

ОПЕРАЦИОННЫЕ СИСТЕМЫ

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

ЛАБОРАТОРНАЯ РАБОТА № 1 Функции получения системной информации

Цель работы:

Получение практических навыков по программированию в Win32 API с использованием аппаратных и системных функций.

Методические указания к выполнению лабораторной работы

Ознакомится с основными Win32 API-функциями и примерами получения системной информации:

1. GetComputerName 2. GetSystemMetrics 3. GetWindowsDirectory 4. GetKeyboardType 5. GetTempPath 6. SetComputerName 7. GetSysColor 8. GetUserName 9. SetSysColors 10. GetSystemDirectory 11. GetVersion 12. SystemParametersInfo 13. GetSystemInfo 14. GetVersionEx 15. GetUserName

Например, для определения имени компьютера используется следующая функция. Функция GetComputerName используется для получения текущего имени компьютера. Связанная с ней SetComputerName используется для присвоения имени компьютеру. Объявление в VC++ записывается следующим образом: `BOOL GetComputerName( LPTSTR lpBuffer, // Адрес буфера имени. LPDWORD nSize // Размер буфера имени. );` Далее приведено описание функции GetComputerName в Delphi (unit Windows): `function GetComputerName(lpBuffer: PChar; var nSize: DWORD): BOOL; stdcall; function GetComputerName; external kernel32 name 'GetComputerNameA';` Ниже приведен код соответствующей VB-декларации: `Private Declare Function GetComputerName Lib "kernel32" Alias "GetComputerNameA" ( _ ByVal lpBuffer As String, _ nSize As Long _ ) As Long` В соответствии с документацией, выполнение функции GetComputerName в Windows 9x завершится неудачей, если размер буфера входных данных меньше, чем величина константы MAX_COMPUTERNAME_LENGTH + 1. Эта константа определена в WINBASE.H как `#define MAX_COMPUTERNAME_LENGTH 15` в Delphi (unit Windows): `MAX_COMPUTERNAME_LENGTH = 15;` в VB записывается таким образом: `Const MAX_COMPUTERNAME_LENGTH = 15 { фрагмент на Delphi }` `var Bufer : array [0..MAX_COMPUTERNAME_LENGTH+1] of char; size : cardinal; . . . . . size := SizeOf(Bufer); GetComputerName(@Bufer, size); edit1.text:=bufer; ' фрагмент на VB` `Public Function GetTheComputerName() As String Dim s As String, lng As Long s = String(MAX_COMPUTERNAME_LENGTH + 1, 0) lng = GetComputerName(s, MAX_COMPUTERNAME_LENGTH) GetTheComputerName = s End Function` Задание для выполнения лабораторной работы 1 Разработать программу, обеспечивающую получение следующей системной информации: 1. Имя компьютера, имя пользователя. 2. Пути к системным каталогам Windows. 3. Версия операционной системы. 4. Системные метрики (3 системные метрики по вариантам) 5. Системные параметры (3 системных параметра): 6. Системные цвета (определить цвет для символьных констант и изменить его на любой другой):

Вариант 3:

Параметр nIndex

COLOR_BTNFACE, COLOR_GRAYTEXT, COLOR_DESKTOP

Функции для работы со временем (Название API-функции):

GetSystemTime, GetTimeZoneInformation, EnumTimeFormats

Дополнительные API-функции:

AnsiToOemBuff, GetCursorPos, GetNumberFormat, SetCaretPos

Ход выполнения лабораторной работы

1. В соответствии с заданием определить необходимые системные функции; 2. Для полученного ряда функций найти определения в зависимости от используемого языка программирования. Поиск определений функций можно произвести на сайте pinvoke.net; 1. Найденные декларации перенести в код проекта;

2. Изучить описание необходимых функций, особенности их параметров и возвращаемых результатов

3. Исчерпывающую информацию о системных функциях, системных структурах и т.д. можно получить на сайте msdn.microsoft.com;

4. В соответствии с заданием реализовать системные функции.

Код программы:

```
// Получение системной информации
void PrintSystemInfo() {
    char computerName[MAX_COMPUTERNAME_LENGTH + 1];
    DWORD size = sizeof(computerName);
    GetComputerNameA(computerName, &size);
    cout << "Computer name: " << computerName << endl;

    char userName[UNLEN + 1];
    size = sizeof(userName);
    GetUserNameA(userName, &size);
    cout << "User name: " << userName << endl;

    char systemDir[MAX_PATH];
    GetSystemDirectoryA(systemDir, MAX_PATH);
    cout << "System directory: " << systemDir << endl;

    char windowsDir[MAX_PATH];
    GetWindowsDirectoryA(windowsDir, MAX_PATH);
    cout << "Windows directory: " << windowsDir << endl;

    char programFilesDir[MAX_PATH];
    SHGetFolderPath(NULL, CSIDL_PROGRAM_FILES, NULL, 0, programFilesDir);
    cout << "Program Files directory: " << programFilesDir << endl;

    OSVERSIONINFO osv;
    osv.dwOSVersionInfoSize = sizeof(OSVERSIONINFO);
    GetVersionEx(&osv);
    cout << "OS version: " << osv.dwMajorVersion << "." << osv.dwMinorVersion << endl;
}

int GetSystemMetric(int index) {
    return GetSystemMetrics(index);
}

// Функции для работы с системными цветами
void PrintSysColors() {
    int elements[] = { COLOR_BTNFACE, COLOR_GRAYTEXT, COLOR_DESKTOP };
    for (int element : elements) {
        cout << "Color " << element << ": " << GetSysColor(element) << endl;
    }
}

void SetSysColors() {
    int elements[] = { COLOR_BTNFACE, COLOR_GRAYTEXT, COLOR_DESKTOP };
    COLORREF colors[] = { RGB(0, 255, 0), RGB(0, 0, 255), RGB(255, 0, 0) };
    SetSysColors(3, elements, colors);
}

void RestoreSysColors(COLORREF originalColors[]) {
    int elements[] = { COLOR_BTNFACE, COLOR_GRAYTEXT, COLOR_DESKTOP };
    SetSysColors(3, elements, originalColors);
}
```

```
int main() {
    // Сохранение текущих значений системных цветов
    COLORREF originalColors[3];
    int elements[] = { COLOR_BTNFACE, COLOR_GRAYTEXT, COLOR_DESKTOP };
    for (int i = 0; i < 3; ++i) {
        originalColors[i] = GetSysColor(elements[i]);
    }

    // Получение системной информации
    PrintSystemInfo();

    // Получение системных метрик
    cout << "Screen resolution: " << GetSystemMetric(SM_CXSCREEN) << "x" << GetSystemMetric(SM_CYSCREEN) << endl;
    cout << "Number of mouse buttons: " << GetSystemMetric(SM_MOUSEBUTTONS) << endl;

    // Получение системных параметров
    cout << "Double click time: " << GetSystemMetric(SM_CXDOUBLECLK) << endl;
    cout << "Drag height: " << GetSystemMetric(SM_CYDRAG) << endl;
    cout << "Show sounds: " << GetSystemMetric(SM_SHOWSOUNDS) << endl;

    // Системные цвета
    PrintSysColors();
    SetSysColors();
    cout << "Colors changed." << endl;

    // Дополнительные функции
    PrintCursorPos();
    SetCursorToPosition(100, 100);

    // Восстановление системных цветов
    RestoreSysColors(originalColors);
    cout << "Colors restored." << endl;

    return 0;
}
```

```

void PrintTimeZoneInformation() {
    TIME_ZONE_INFORMATION tzi;
    GetTimeZoneInformation(&tzi);
    wcout << L"Time zone: " << tzi.StandardName << endl;
}

BOOL CALLBACK EnumTimeFormatsProc(LPTSTR lpTimeFormatString) {
    wcout << L"Time format: " << lpTimeFormatString << endl;
    return TRUE; // Продолжить перечисление
}

void EnumTimeFormats() {
    EnumTimeFormats(EnumTimeFormatsProc, 0, 0);
}

// Функции для работы с курсором и позиционированием
void PrintCursorPos() {
    POINT pt;
    GetCursorPos(&pt);
    cout << "Cursor position: X=" << pt.x << ", Y=" << pt.y << endl;
}

void SetCursorToPosition(int x, int y) {
    SetCaretPos(x, y);
    cout << "Caret position set to (" << x << ", " << y << ")" << endl;
}

#include <iostream>
#include <windows.h>
#include <lmcons.h> // Для UNLEN
#include <shlobj.h> // Для SHGetFolderPath
#include <tchar.h> // Для TCHAR и связанных функций
using namespace std;

// Предварительные объявления функций
void PrintSystemTime();
void PrintTimeZoneInformation();
BOOL CALLBACK EnumTimeFormatsProc(LPTSTR lpTimeFormatString);
void EnumTimeFormats();
void PrintCursorPos();
void SetCursorToPosition(int x, int y);
void PrintSystemInfo();
int GetSystemMetric(int index);
void PrintSysColors();
void SetSysColors();
void RestoreSysColors(COLORREF originalColors[]);

// Структуры для работы с системным временем
void PrintSystemTime() {
    SYSTEMTIME st;
    GetSystemTime(&st);
    cout << "System time: " << st.wHour << ":" << st.wMinute << ":" << st.wSecond << endl;
}

```

Результат работы программы:

The screenshot shows a Windows command prompt window titled "C:\Documents and Settings\Льва Шеня\Рабочий стол\Untitled1.exe". The window displays the following output:

```

Computer name: DEBUGGER
User name: Л\ь\ш\ш\я\я\
System directory: C:\WINDOWS\system32
Windows directory: C:\WINDOWS
Program Files directory: C:\Program Files
OS version: 5.1
Screen resolution: 1097x539
Number of mouse buttons: 5
Double click time: 4
Drag height: 4
Show sounds: 0
Color 15: 14215660
Color 17: 10070188
Color 1: 9981440
Colors changed.
System time: 17:31:20
Time zone: Cursor position: X=461, Y=308
Caret position set to (100, 100)
Colors restored.

Process returned 0 (0x0)    execution time : 0.125 s
Press any key to continue.

```